

Contents

Implementation Plan: Spatially-Varying Quasi-Dynamic Earthquake-Cycle Driver	1
Overview	1
Constraints	2
Phase 0: Friction-config TOML schema + parser (no code yet — schema spec)	3
Phase 1: SAFS friction-config parser + per-DOF resolver	6
Phase 2: Velocity-sidecar bundle (multi-model)	9
Phase 3: Stress-sidecar bundle	11
Phase 4: <code>seas_spatial_qd_driver</code> — main driver	12
Phase 5: Operator accessor additions (minimal additive surface)	15
Phase 6: Sbatch + run script + minimal verification	16
Testing Strategy	17
Risk Assessment	18
Out of Scope for This Plan	19

Implementation Plan: Spatially-Varying Quasi-Dynamic Earthquake-Cycle Driver

Scope generalisation note (2026-05-17): this driver is generic — any simulation with spatially-varying material / pre-stress / friction can use it. The SAFS multi-fault San Andreas dataset under `safs/project_7.0_alternative/...` is the first concrete example exercised by the acceptance tests; the architecture and CLI carry no SAFS-specific assumptions. All driver, source, and test names use the `spatial` prefix; only dataset paths and SAFS-specific TOML/sbatch fills retain the `safs` infix to disambiguate which example dataset they target.

Date: 2026-05-17 Supersedes (in part): `heterogeneous_material_plan.md` — that plan defined the cross-cutting `MaterialField::Mode::Coefficient` and `ElasticityDomainOperator(MaterialField)` constructor and is prerequisite, not duplicated here. This plan layers driver wiring, restart, and ParaView output on top. Companion plan: `spatial_dynamic_rupture_plan.md` (dynamic-rupture sibling that shares the same mesh / velocity / stress / friction data layout).

Overview

Build a single driver `seas_spatial_qd_driver` that runs the SCEC SEAS sequences-of-earthquakes-and-aseismic-slip (SEAS) quasi-dynamic problem on the SAFS multi-fault San Andreas geometry with all four data layers spatially varying: mesh (`safs/project_7.0_alternative/meshing/results/msh/*.msh`), velocity (`safs/project_7.0_alternative/velocity/results/<model>/*.h5` sidecar), pre-stress (`safs/project_7.0_alternative/stress/results/*.h5` sidecar), and rate-and-state friction parameters (`safs/project_7.0_alternative/friction/*.toml`, files to be supplied).

The driver reuses every existing BP5 building block (`ElasticityDomainOperator`, `RateStateFaultOperator<ParMesh, 2>`, `SEASQuasiDynamicOperator`, RK45 / PETSc-TS time stepping, `ParaViewOutput`, V2 checkpoint / `--restart`) and adds only the wiring needed to:

1. Construct `MaterialField::Mode::Coefficient` from the velocity sidecar (CVM-H / CVM-S4.26.M01 / multi-scale-statewise) — extending the work in `heterogeneous_material_plan.md`.
2. Build per-DOF fault pre-stress from `StressField3D` via the already-implemented `FaultGeometry::ComputeSAFSParams` (Phase 6 §5 of `PLAN_onfaultstress.md`).
3. Resolve per-DOF rate-and-state parameters (`a`, `b`, `Dc`, `V_init`, `f_0`, `V_0`, `eta`, `sigma_n_eff`) from a TOML friction-config sidecar with depth-/region-based spatial laws.

4. Wire ParaViewOutput to the new VTKHDF + ZFP back end from the paraview-compaction merge (single <prefix>/volume.vtkhdf, single <prefix>/fault.vtkhdf, adaptive snapshot cap, regime-aware cadence).
5. Use the V2 PETSc-TS restart path (io/petsc_ts_checkpoint.hpp) for long earthquake-cycle runs that exceed a single Frontera job's wall-clock.

Quasi-dynamic-specific scope: this driver uses the radiation-damping approximation ($\tau_{\text{radiation}} = \eta \cdot V$, no inertia) and runs on the order of decades to centuries of simulated time with adaptive Δt that grows in interseismic periods (years) and shrinks in coseismic phase (seconds). It does not solve a wave equation; that is spatial_dynamic_rupture_plan.md's scope.

Constraints

Interface constraints — what cannot change

- **ElasticityDomainOperator<ParMesh>** two existing constructors (legacy (λ , μ) and MaterialField-aware) remain bit-exact for non-SAFS callers. This driver uses the MaterialField constructor exclusively; if the heterogeneous_material_plan Phase 2 is not yet merged this driver will not link.
- **RateStateFaultOperator<ParMesh, 2>::SetSAFSMode(true, ...)** is the only mechanism for enabling per-DOF tau_pre / sigma_n. R-001 guard requires geom_->NumZeroNormalFallbacks() == 0; the driver MUST inspect this before enabling.
- **FaultGeometry::ComputeSAFSParams(field, P_p_pa, P_p_grad, min_sigma_n_pa)** is the only path to populate per-DOF pre-stress; do not duplicate the projection.
- **SEASQuasiDynamicOperator<ParMesh, ElasticityDomainOperator<ParMesh>, BP5FaultOp>** template (alias BP5SEASOp in solver/seas_operator.hpp:551) is the time-integration coupling shell; do not subclass.
- **ParaViewOutput<ParMesh>** API: RegisterDomainField, InitFaultOutputBP5(interior_faces, shared_faces, nbf_per_face), SetFaultParamsBP5, UpdateFaultFieldsBP5(slip, slip_rate, traction, state, normal_stress), SetFaultOutputMode(FaultOutputMode::Hdf5), SetHDFCompression(...). Do not re-implement.
- Restart V2 schema(io/petsc_ts_checkpoint.hpp) is fixed. Driver writes <output>/checkpoint.h5 with one HDF5 group per (rank × stride) — see existing BP5 V2 + TPV104 V1 restart commit (63373f8) for the contract.
- CLAUDE.md sign conventions apply unchanged: slip-rate || traction; tau_pre || V_init; dip = (0, 0, +1) downward; $\sigma_n > 0$ = compression. The SAFS fault is now curvilinear (multi-segment SAFS-MJVS-SBMT-MULT) but the local FaultBasis on each DOF still produces (n, t1=dip, t2=strike) with the same sign-flip convention.

Dependency constraints

- **heterogeneous_material_plan.md** Phase 1+2 must be merged first — the driver uses MaterialField::MakeCoefficient(...) and the ElasticityDomainOperator(MaterialField) constructor. If the plan is not yet implemented, this driver is implementable but cannot run heterogeneous; in that case it falls back to MaterialField::MakeConstant(mu_const, lambda_const, rho_const) with values from the friction-config TOML's [material_constant_fallback] section.
- **StressField3D** (io/stress_field_3d.hpp) already exists and is exercised by seas_test_compute_safs_params (13/13 pass). The driver reuses it verbatim.
- **DataField3D** + FieldCoefficient (existing) provide the velocity-sidecar trilinear path. No changes.
- MFEM build flags: MFEM_USE_HDF5=YES (for VTKHDF), MFEM_USE_PETSC=YES (for PETSc-TS restart), MFEM_USE_H5Z_ZFP=YES (optional, for ZFP compression). setup_mfem.sh auto-enables HDF5 + PETSc when conda env has them; ZFP requires manual make config MFEM_USE_H5Z_ZFP=YES.

- **miniapps/seas/CLAUDE.md** rules: do NOT modify `friction/dieterich_ruina.hpp`, `solver/seas_operator.h`, `bp5/`, `bp1/`, `bp2/`. Driver lives entirely under `drivers/`; helpers under `spatial/code/`.

Convention constraints

- TOML for all configuration. Driver accepts `--config PATH.toml` exclusively — no positional CLI flags for physical parameters. Naming pattern follows existing `config/bp5_production_new_driver.toml`. CLI is reserved for runtime knobs (`--restart`, `--paraview-*`, `--output-dir`, `--tfinal`, `--np`).
- Output directory layout mirrors paraview-compaction Phase 6 conventions:


```
<output_prefix>/
  volume.vtkhdf          (or volume/Cycle*/proc*.vtu if --paraview-volume-vtu)
  fault.vtkhdf           (or fault/Cycle*/data*.vtu if --paraview-fault-vtu)
  stations/              (probe output: <station_label>.txt per station)
  checkpoint.h5          (V2 PETSc-TS checkpoint; one per --checkpoint-every)
  seas_spatial_qd_driver.log (driver stdout/stderr tee)
```
- All physical units SI. Sidecars store stress in Pa, slip in m, velocity-loading in m/s, time in s; the only exception is human-readable `tfinal` in TOML which accepts suffix-coded values ("300yr", "5e9s", "1Ma") parsed by `spatial/code/spatial_time_parser.hpp` (Phase 1).
- Naming: source files `safs_qd_*.cpp/hpp` under `spatial/code/`; test files `tests/unit/test_safs_qd_*.cpp`; sbatch under `jobs/safs/`.
- Greppable markers: every TODO this plan defers is tagged `// SPATIAL-QD R-XXX TODO`.

Numerical constraints

- Quasi-dynamic time-integration tolerances: `atol = 1e-7`, `rtol = 1e-50` (pure-absolute, same as BP5 production). Adaptive RK45 (or PETSc-TS RK4/5 when `--petsc-ts` set).
- Max Δt : $0.1 * \text{seconds_per_year}$ ($= 3.155693e6$ s) for SCEC output compliance. Hard-coded into `TimeConfig` defaults.
- Initial Δt : $0.01 * L_{\text{nuc}} / V_{\text{nuc}}$ where $L_{\text{nuc}} = \mu * D_c / (b * \sigma_{\text{n_eff}})$ from the deepest fault DOF in the nucleation zone. Computed at runtime from the loaded sidecar values, NOT hardcoded.
- Friction equilibrium tolerance: post-init, $\max_i |F(V_i, \psi_i)| < 1e-6 * V_{\text{init_max}}$ across all fault DOFs. Reuses BP5's `InitialStatePsi` Brent solver.
- Per-DOF heterogeneity: all of `a`, `b`, `Dc`, `V_init`, `f_0`, `V_0`, `eta`, `sigma_n_eff`, `tau_pre` are per-DOF vectors of length `num_fault_dofs`. No scalar BP5-style cache values. `RateStateFault0perator::SetSAFSMode(true, ...)` is the wiring point.
- MPI determinism: `MPI_Allreduce(MPI_MAX)` on RK45 local error (preserve BP5 v9 fix that prevents rank-divergence deadlock).

Phase 0: Friction-config TOML schema + parser (no code yet — schema spec)

Goal

Define the exact TOML schema for `friction/<run-name>.toml` so the user can drop in values without further plan changes. Schema lives in `safs/project_7.0_alternative/document/spatial_friction_config_schema` (also written by this phase) and is consumed by Phase 1.

Files to Create

- `safs/project_7.0_alternative/document/spatial_friction_config_schema.md` — mark-down schema reference with rationale, units, and an example TOML for both rate-and-state and slip-weakening fillings. ≈ 200 LOC.
- `safs/project_7.0_alternative/friction/EXAMPLE_spatial_friction_rate_state_safs.toml` — concrete example file (Aging-law DRS, BP5-style scalar fallback + SAFS-style per-region overrides) for the user to copy-edit.
- `safs/project_7.0_alternative/friction/EXAMPLE_spatial_friction_slip_weakening_safs.toml` — concrete example file (LSW with $\mu_s / \mu_d / d_c$ by region) for the dynamic-rupture sibling plan; included here so the friction directory is self-contained.

Detailed Requirements

1. Schema top-level (TOML):

```
[meta]
schema_version = 1          # mandatory; aborts if mismatch
law             = "rate_state" # "rate_state" | "slip_weakening"
description     = "SAFS-ALT6 multi-fault rate-and-state Aging law"

[material_constant_fallback]
# Used if --no-sidecar-material is passed OR sidecar is absent.
lambda = 32.0e9 # Pa
mu      = 32.0e9 # Pa
rho     = 2670.0 # kg/m^3

[pore_pressure]
# Effective normal stress =  $\sigma_{n\_total} - P_p$  (constant + depth gradient)
P_p_pa      = 0.0
P_p_grad_pa_per_m = 0.0
min_sigma_n_pa = 0.0 # 0 = no clamp; >0 = MFEM_VERIFY guard floor

[time]
tfinal      = "300yr" # accepts s | yr | ka | Ma suffix (parsed by safs_time_parser)
t_initial   = 0.0
dt_initial  = "auto" # "auto" => 0.01 * L_nuc / V_nuc from loaded params
dt_max      = "0.1yr"
atol        = 1.0e-7
rtol        = 1.0e-50

[output]
output_dir      = "output_safs_qd_500m"
tfinal_output_dt = "0.01yr" # cap on inter-seismic write cadence
coseismic_dt    = "0.01s" # coseismic event-detection cadence
max_snapshots   = 5000 # adaptive cap
paraview_volume = "hdf5" # "hdf5" | "vtu" | "off"
paraview_fault  = "hdf5" # "hdf5" | "vtu" | "off"
paraview_volume_zfp_tol = 1.0e-3 # m (displacement)
paraview_fault_zfp_tol  = 1.0e-12 # m/s (slip rate floor)
checkpoint_every_steps = 5000 # V2 PETSc-TS write cadence
```

2. Rate-and-state-specific block ([`friction.rate_state`]): required when `meta.law = "rate_state"`.

```
[friction.rate_state]
# Constants (scalars, applied as defaults everywhere).
```

```

f_0      = 0.6           # reference friction coefficient
V_0      = 1.0e-6        # reference slip rate [m/s]
eta      = 0.5 * sqrt(mu * rho) / mu    # radiation damping; auto-computed if "auto"
                                           # (string "auto" -> eta = 0.5 * sqrt(mu * rho), evaluated per-DOF
                                           # using EvalAt on the fault element)

# Spatial fields - default scalar, then per-region / per-depth overrides.
a_default = 0.010
b_default = 0.015
Dc_default = 0.004      # m
V_init_default = 1.0e-9 # m/s
sigma_n_default = 50.0e6 # Pa

# Spatial heterogeneity rules (applied in order, last match wins).
# "depth": triggers when DOF z is inside [z_min, z_max] (z negative = below surface)
# "region_attribute": triggers when DOF lies on a Physical Surface with given gmsh tag
# "box": triggers when (x, y, z) lies inside an axis-aligned box
[[friction.rate_state.spatial]]
kind      = "depth"
z_min_m   = -5000.0
z_max_m   = 0.0
a          = 0.0125    # shallow strengthening
b          = 0.013
Dc         = 0.0085

[[friction.rate_state.spatial]]
kind      = "depth"
z_min_m   = -15000.0
z_max_m   = -5000.0
a          = 0.005      # seismogenic VW core
b          = 0.015

[[friction.rate_state.spatial]]
kind      = "box"
x_min_m   = ...        # nucleation patch (user fills)
x_max_m   = ...
y_min_m   = ...
y_max_m   = ...
z_min_m   = ...
z_max_m   = ...
V_init     = 1.0e-2     # nucleation forcing

```

3. Slip-weakening-specific block (**[friction.slip_weakening]**): required when `meta.law = "slip_weakening"`. Same `[[spatial]]` array pattern but with keys `mu_s`, `mu_d`, `d_c`, `cohesion`. Detailed contents are in `spatial_dynamic_rupture_plan.md` Phase 1 (this plan's sibling); the example file `EXAMPLE_spatial_friction_slip_weakening_safs.toml` shows the layout.

4. Validation rules (all enforced in Phase 1's parser):

- `meta.schema_version == 1` — MFEM_ABORT otherwise.
- `meta.law ∈ {"rate_state", "slip_weakening"}` — MFEM_ABORT otherwise.
- In rate-state law: $0 < a < b < 0.1$, $D_c > 0$, $V_0 > 0$, $\sigma_n > 0$, $0 < f_0 < 1$ at every DOF after spatial application.
- In slip-weakening: $0 < \mu_d < \mu_s < 1$, $d_c > 0$ at every DOF.
- `pore_pressure.min_sigma_n_pa ≥ 0`.
- `time.tfinal` parsed successfully (otherwise MFEM_ABORT with the offending string).

Acceptance Criteria

- ☐ spatial_friction_config_schema.md documents every key, units, and validation rule.
- ☐ The two EXAMPLE_*.toml files load through Phase 1's parser (added once Phase 1 lands).
- ☐ Schema is reviewed and signed off by the user before Phase 1 begins (this is a contract).

Dependencies

- Depends on: nothing (pure spec).
 - Required by: Phase 1.
-

Phase 1: SAFS friction-config parser + per-DOF resolver

Goal

Implement SpatialFrictionConfig (struct + parser) that reads a Phase-0-schema TOML and SpatialFrictionResolver that maps (fault_DOF_index, x, y, z, gmsl_region_attr) → (a, b, Dc, V_init, f_0, V_0, eta, sigma_n_eff, tau_pre) for rate-and-state, or (mu_s, mu_d, d_c, cohesion, tau_pre) for slip-weakening. Per-DOF vectors are produced once at driver init.

Files to Create

- spatial/code/spatial_friction_config.hpp / .cpp — TOML parser using existing extern/toml11 (already a dependency, see miniapps/seas/Makefile:SEAS_USE_TOML).
- spatial/code/spatial_friction_resolver.hpp / .cpp — applies spatial rules.
- spatial/code/spatial_time_parser.hpp / .cpp — small helper for "300yr" / "5Ma" / "0.01s" parsing.
- tests/unit/test_spatial_friction_config.cpp — TOML round-trip + validation tests.
- tests/unit/test_spatial_friction_resolver.cpp — spatial-rule evaluation tests.

Interfaces

```
namespace mfem::seas::spatial {  
  
enum class FrictionLawKind { RateState, SlipWeakening };  
  
struct PorePressureSpec {  
    real_t P_p_pa          = 0.0;  
    real_t P_p_grad_pa_per_m = 0.0;  
    real_t min_sigma_n_pa    = 0.0;  
};  
  
struct MaterialConstantFallback {  
    real_t lambda = 32.0e9;  
    real_t mu      = 32.0e9;  
    real_t rho     = 2670.0;  
};  
  
// Holds either rate_state or slip_weakening table, not both.  
struct SpatialFrictionConfig {  
    int          schema_version = 0;  
    FrictionLawKind law         = FrictionLawKind::RateState;  
    std::string  description;  
};
```

```

MaterialConstantFallback material_fallback;
PorePressureSpec         pore_pressure;
// Time + output configs as nested structs (omitted here for brevity).
TimeConfig               time;
OutputConfig             output;

// Friction-law-specific:
struct RateStateBlock {
    real_t f_0_default = 0.6;
    real_t V_0_default = 1.0e-6;
    // "auto" → computed per-DOF as 0.5 * sqrt(mu * rho) using MaterialField.
    bool eta_auto_from_material = true;
    real_t eta_default = 0.0;
    real_t a_default, b_default, Dc_default, V_init_default, sigma_n_default;
    std::vector<SpatialRule> spatial; // applied in order, last-match wins
};
struct SlipWeakeningBlock {
    real_t mu_s_default, mu_d_default, d_c_default, cohesion_default;
    std::vector<SpatialRule> spatial;
};
std::optional<RateStateBlock> rate_state; // populated iff law == RateState
std::optional<SlipWeakeningBlock> slip_weakening; // populated iff law == SlipWeakening
};

// Parse + validate. Aborts on any schema violation with a precise message.
SpatialFrictionConfig LoadSpatialFrictionConfig(const std::string& tom_path);

// Resolved per-DOF parameter vectors for rate-and-state law.
struct RateStatePerDOFParams {
    Vector a; // [num_fault_dofs]
    Vector b;
    Vector Dc;
    Vector V_init; // scalar magnitude; direction comes from tau_pre orientation
    Vector f_0;
    Vector V_0;
    Vector eta;
    Vector sigma_n_eff; // ( $\sigma_n_{total} - P_p$ ) clamped by min_sigma_n_pa
    // Note: tau_pre comes from FaultGeometry::ComputeSAFSPParams, NOT this struct.
};

// Resolved per-DOF parameter vectors for slip-weakening law.
struct SlipWeakeningPerDOFParams {
    Vector mu_s;
    Vector mu_d;
    Vector d_c;
    Vector cohesion;
};

class SpatialFrictionResolver {
public:
    // Build per-DOF params by walking every owned fault DOF, applying
    // the spatial rules in order, and computing eta from MaterialField
    // (Mode::Coefficient via EvalAt(elem, T, ip)) if eta_auto.
    //

```

```

// dof_coords_3d: interleaved (x_i, y_i, z_i) of length 3 * num_dofs
// dof_to_elem:   [num_dofs] local bulk-element index that owns the DOF
//               (typically the elem1 side of the fault face)
// dof_to_attr:   [num_dofs] gmsh Physical Surface attribute at each DOF
//               (= 101 for "fault" in the SAFS .geo)
// material:      used only when eta is "auto"
// mesh:          used only when eta is "auto" (for ElementTransformation)
RateStatePerDOFParams ResolveRateState(
    const SpatialFrictionConfig::RateStateBlock& cfg,
    const Vector& dof_coords_3d,
    const Array<int>& dof_to_elem,
    const Array<int>& dof_to_attr,
    const MaterialField& material,
    ParMesh& mesh) const;

SlipWeakeningPerDOFParams ResolveSlipWeakening(
    const SpatialFrictionConfig::SlipWeakeningBlock& cfg,
    const Vector& dof_coords_3d,
    const Array<int>& dof_to_elem,
    const Array<int>& dof_to_attr) const;
};

} // namespace mfem::seas::spatial

```

Detailed Requirements

1. **SpatialRule** discriminated union:

```

struct SpatialRule {
    enum class Kind { Depth, Box, RegionAttribute };
    Kind kind;
    // Depth: only z_min / z_max used
    // Box:   all 6 used
    // RegionAttribute: only region_attr used
    real_t x_min_m = -std::numeric_limits<real_t>::infinity();
    real_t x_max_m =  std::numeric_limits<real_t>::infinity();
    real_t y_min_m = -std::numeric_limits<real_t>::infinity();
    real_t y_max_m =  std::numeric_limits<real_t>::infinity();
    real_t z_min_m = -std::numeric_limits<real_t>::infinity();
    real_t z_max_m =  std::numeric_limits<real_t>::infinity();
    int    region_attr = -1;
    // Per-key overrides (NaN = "don't override, keep default or prior rule").
    real_t a = nan, b = nan, Dc = nan, V_init = nan;
    real_t sigma_n = nan, f_0 = nan, V_0 = nan, eta = nan;
    real_t mu_s = nan, mu_d = nan, d_c = nan, cohesion = nan;
};

```

A rule “matches” a DOF when the relevant inequalities all hold; non-NaN overrides are then applied. Rules apply in document order; last match wins per key.

2. **ResolveRateState** algorithm (per DOF i):

1. Seed $a_i = \text{cfg.a_default}$, $b_i = \text{cfg.b_default}$, ..., $\sigma_{n,i} = \text{cfg.sigma_n_default}$.
2. For each rule r in cfg.spatial (in order):
 - if rule matches DOF i :
 - for each non-NaN override key k , set $k_i = \text{rule.k}$

3. Compute $\sigma_{n_eff_i} = \max(\min_sigma_n_pa, \sigma_{n_i} - (P_p_pa + P_p_grad_pa_per_m * \max(0, -z_i)))$
4. If `cfg.eta_auto_from_material`:
 - Get `T = mesh.GetElementTransformation(dof_to_elem[i])`
 - `ip` ← reference centroid of `T`
 - `EvalAt(material, dof_to_elem[i], *T, ip, lambda_e, mu_e, rho_e)`
 - `eta_i = 0.5 * sqrt(mu_e * rho_e)`
 - else: `eta_i = cfg.eta_default`
5. Validate: $0 < a_i, 0 < b_i, a_i < b_i, Dc_i > 0, V_{0_i} > 0, \sigma_{n_eff_i} > 0, 0 < f_{0_i} < 1$. MFEM_VERIFY on each.

3. **SpatialFrictionConfig** validation runs at end of `LoadSpatialFrictionConfig`:

- `schema_version == 1`
- law parseable to `FrictionLawKind`
- Either `rate_state` or `slip_weakening` populated (but not both); the non-matching one MUST be absent in the TOML (extra keys → MFEM_ABORT).
- For rate-state: defaults pass the per-DOF validation rule above (with `sigma_n_default` as the default σ_{n_eff} input).

4. **safs_time_parser**: simple regex `^[+-]?[0-9.eE+-]+\s*(s|yr|ka|Ma)?$`; conversion table `s=1, yr=365.25*86400, ka=1e3*yr, Ma=1e6*yr`. Returns `real_t` seconds.

Acceptance Criteria

- ☐ `test_spatial_friction_config`: 10+ tests covering `schema_version` mismatch, missing law, both laws present, unknown keys, every validation rule.
- ☐ `test_spatial_friction_resolver`: 8+ tests on synthetic 12-DOF fault — depth-only rule, box rule, region rule, layered (multiple rules), `eta_auto` vs `eta_default`.
- ☐ Two `EXAMPLE_*.toml` files parse without error.

Dependencies

- Depends on: Phase 0 schema; `extern/toml11`; `heterogeneous_material_plan.md` Phase 1 (`MaterialField::Mode::Coefficient`) for the `eta_auto` path.
- Required by: Phase 4.

Phase 2: Velocity-sidecar bundle (multi-model)

Goal

Wrap `heterogeneous_material_plan.md`'s `SidecarMaterialBundle` to accept any of the three SAFS velocity models — CVM-H, CVM-S4.26.M01, multi-scale-statewise — via a single `--velocity-model` TOML key, resolving the actual sidecar HDF5 path from the canonical directory layout.

Files to Create

- `spatial/code/spatial_velocity_bundle.hpp / .cpp` — thin layer over `LoadSidecarMaterialBundle`.

Interfaces

```
namespace mfem::seas::spatial {
```

```

enum class VelocityModel { CVMH, CVMS_4_26_M01, MultiscaleStatewise };

struct SpatialVelocitySpec {
    VelocityModel model      = VelocityModel::CVMH;
    std::string  mesh_tag    = "1000m_lcfar3000"; // matches mesh basename
    std::string  override_path; // "" → resolve from model+mesh_tag
};

// Resolve `<dataset_root>/velocity/results/<model_dir>/velocity_<mesh_tag>.h5`.
// (SAFS example fills produce filenames like `velocity_safs_500m.h5`. )
// Override path bypasses resolution.
std::string ResolveSpatialVelocitySidecarPath(const SpatialVelocitySpec& spec,
                                              const std::string& dataset_root);

// Build a SidecarMaterialBundle that owns DataField3D Vp, Vs, density and
// LambdaFromSidecar / MuFromSidecar / RhoFromSidecar Coefficients.
// Performs ContainsBBox check against `pmesh` for all three fields.
SidecarMaterialBundle LoadSpatialVelocityBundle(const SpatialVelocitySpec& spec,
                                              const std::string& dataset_root,
                                              ParMesh& pmesh);

} // namespace mfem::seas::spatial

```

Detailed Requirements

1. Path resolution rule:

- VelocityModel::CVMH → <dataset_root>/velocity/results/cvmh/velocity_safs_<mesh_tag>.h5
- VelocityModel::CVMS_4_26_M01 → <dataset_root>/velocity/results/cvm_s4.26.m01/velocity_safs_<mesh_tag>.h5
- VelocityModel::MultiscaleStatewise → <dataset_root>/velocity/results/multiscale_statewise_<mesh_tag>.h5
- If override_path is non-empty, use it verbatim.

2. MFEM_VERIFY the resolved file exists before passing to DataField3D (so the user gets a clean “file not found at ” rather than the HDF5 library’s terse message).

3. All three fields (Vp, Vs, density) must pass ContainsBBox against the parmesh. Reuse FieldProjector::AbortContainmentFailure — do not roll a new abort path (this matches heterogeneous_material_plan.md R-009 round-3).

4. TOML key mapping for the driver’s [velocity] block:

```

[velocity]
model      = "cvmh"           # cvmh | cvm_s4.26.m01 | multiscale_statewise
mesh_tag    = "1000m_lcfar3000"
override_path = ""           # set non-empty to bypass model+tag resolution

```

Acceptance Criteria

- ☐ test_spatial_velocity_bundle: 4 tests — each model resolves the right path, override works, missing file aborts cleanly.
- ☐ Smoke test: LoadSpatialVelocityBundle on the 1000m_lcfar3000 mesh with the cvmh sidecar succeeds; bundle→MakeMaterialField() returns a MaterialField with mode == Coefficient and all three coefficient pointers non-null.

Dependencies

- Depends on: heterogeneous_material_plan.md Phase 4 (SidecarMaterialBundle, Load-SidecarMaterialBundle).
 - Required by: Phase 4.
-

Phase 3: Stress-sidecar bundle

Goal

Wrap existing StressField3D loading + FaultGeometry::ComputeSAFSParams into a single helper so the driver has one call to produce per-DOF tau_pre and sigma_n_per_dof_.

Files to Create

- spatial/code/spatial_stress_bundle.hpp / .cpp.

Interfaces

```
namespace mfem::seas::spatial {

struct SpatialStressSpec {
    std::string sidecar_path;           // <dataset_root>/stress/results/stress_csm_safs.h5
    real_t      P_p_pa                 = 0.0;
    real_t      P_p_grad_pa_per_m     = 0.0;
    real_t      min_sigma_n_pa        = 0.0;
};

// Loads the stress sidecar AND calls geom.ComputeSAFSParams. After return,
// geom.HasSAFSParams() == true and geom.sigma_n_per_dof() / tau_pre vector
// are populated. Aborts if any zero-normal fallback would silently
// project to zero (mirrors RateStateFaultOperator::SetSAFSMode R-001 guard).
void ApplySpatialStressSidecar(const SpatialStressSpec& spec,
                               FaultGeometry<ParMesh>& geom);

} // namespace mfem::seas::spatial
```

Detailed Requirements

1. Pre-flight check: assert geom.NumZeroNormalFallbacks() == 0 BEFORE calling ComputeSAFS-Params. If any zero-normal fallback exists, abort with the same precise message R-001 uses (so the SAFS-side error chain is consistent with the operator-side guard).
2. Loading: construct StressField3D field(spec.sidecar_path); call geom.ComputeSAFSParams(field, spec.P_p_pa, spec.P_p_grad_pa_per_m, spec.min_sigma_n_pa).
3. No silent fallback: if the sidecar load or ComputeSAFSParams aborts, do not catch — let the abort propagate.
4. TOML mapping:

```
[stress]
sidecar_path    = "safs/project_7.0_alternative/stress/results/stress_csm_safs.h5"
P_p_pa         = 0.0
```

```
P_p_grad_pa_per_m = 0.0
min_sigma_n_pa     = 0.0
```

Acceptance Criteria

- ☐ test_spatial_stress_bundle: 3 tests — sidecar loads, ComputeSAFSParams populates correctly, zero-normal pre-flight aborts.
- ☐ Smoke: invoking on the existing stress_csm_safs.h5 with the 1000m mesh produces a non-empty geom.sigma_n_per_dof() and geom.HasSAFSParams() == true.

Dependencies

- Depends on: StressField3D (existing, exercised by seas_test_compute_safs_params); FaultGeometry::ComputeSAFSParams (existing); the R-001 guard from the recent REVIEW.md fix.
 - Required by: Phase 4.
-

Phase 4: **seas_spatial_qd_driver** — main driver

Goal

Single executable that ties Phases 1-3 + heterogeneous material + BP5 SEAS coupling + ParaView output + V2 restart into one runnable program.

Files to Create

- drivers/spatial_qd_driver.cpp — the main. Layout mirrors seas_driver.cpp (BP5) and tpv104_driver.cpp (paraview/restart wiring) but uses SAFS bundles. ~1200 LOC expected.
- spatial/code/spatial_qd_driver_init.hpp / .cpp — extracted init helpers (parse TOML, build bundles, construct ops, validate equilibrium) so the main() body stays under 300 LOC.

Files to Modify

- miniapps/seas/Makefile:
 - Add SPATIAL_QD_DRIVER_SRC = drivers/spatial_qd_driver.cpp.
 - Add object + link rule (must end with \$(SEAS_POST_LINK_DEDUP_RPATH) per Makefile R-005 convention).
 - Add spatial_qd_driver and spatial_qd_driver_init headers/objects under existing SAFS_* block.
 - New umbrella target test-spatial-qd-driver: seas_spatial_qd_driver that builds the driver and runs it in --dry-run mode against a smoke fixture (does NOT run a real simulation — that's an sbatch job).
- miniapps/seas/.gitignore: add seas_spatial_qd_driver to the binary list.
- miniapps/seas/safs/project_7.0_alternative/document/spatial_friction_config_schema.md: cross-reference this driver as the consumer (link added once both files exist).

Interfaces

The driver's CLI:

```
seas_spatial_qd_driver --config PATH.toml [OPTIONS]
```

Required:

```
--config PATH.toml          SAFS QD configuration
```

Mesh/data overrides (otherwise read from TOML):

```
--mesh PATH.msh           Override [mesh].path
--velocity-model {cvmh|cvm_s4.26.m01|multiscale_statewise|<path>}
--stress-sidecar PATH.h5
--friction-config PATH.toml
```

Runtime knobs:

```
--tfinal SECONDS|"300yr"  Override [time].tfinal
--output-dir DIR           Override [output].output_dir
--restart PREFIX           Resume from <PREFIX>/checkpoint.h5 (V2 PETSc-TS)
--checkpoint-every N       Override [output].checkpoint_every_steps
--petsc-ts                 Use PETSc-TS instead of MFEM RK45
```

ParaView (override [output]):

```
--paraview-volume {hdf5|vtu|off}
--paraview-fault {hdf5|vtu|off}
--paraview-volume-zfp-tol TOL
--paraview-fault-zfp-tol TOL
--paraview-max-snapshots N
--no-volume-pv             Shortcut for --paraview-volume=off
```

Diagnostics:

```
--dry-run                 Parse + construct + validate equilibrium; do not time-step
--verify-dispatch         Print per-rank dispatch tags + exit before time loop
--print-derived            After init, print L_nuc, T_nuc, V_nuc, dt_initial per region
```

Detailed Requirements

The driver main() follows this exact order. Each step is greppable:

1. MPI init — same as `tpv104_driver.cpp`.
2. CLI parse — populate a `DriverCLI` struct; merge with the TOML loaded in step 4.
3. Output-dir + restart-dir safety checks — reuse `tpv104_driver.cpp:537–598` `weakly_canonical` logic. Abort if `--output-dir` is inside `--restart` parent or vice-versa.
4. Load **SpatialFrictionConfig** (Phase 1) from `--config`.
5. Load mesh — `Mesh smesh(toml.mesh.path); ParMesh pmesh(MPI_COMM_WORLD, smesh); smesh.Clear();`. Reuse existing `BoundaryConfig` from `safs_test_driver` (`fault_attr = 101`, `dirichlet_attrs = sides + bottom`, `natural_attr = top` — per the `safs_fault_box_nwcut.geo` Physical Surface tags above).
6. Build material bundle (Phase 2) — `auto vbundle = LoadSpatialVelocityBundle(...); MaterialField material = vbundle.MakeMaterialField();` In `--no-sidecar-material` (added below) or if velocity is missing, fall back to `MaterialField::MakeConstant(toml.material_fallback.lambda, ..., toml.material_fallback.rho)` with a one-line `mfem::out` notice.
7. Construct **ElasticityDomainOperator** with the `MaterialField` constructor (heterogeneous_material_plan Phase 2). Use `DGMethod::BR2`, `SolverType::MUMPS_BLR`, `order = toml.numerics.order` (default 1). This is the only ctor used.
8. Construct **FaultGeometry<ParMesh>** `geom(domain_op, bp5_params_seed)` where `bp5_params_seed` is a placeholder `BP5Params{}` — the per-DOF rate-state values from Phase 1 override its scalars before any consumption. The constructor unconditionally calls `ComputePerDOFCoordsAndBasis_` (REVIEW R-001 guard ensures any zero-normal fallback raises a loud warning at this point).

9. Apply stress sidecar (Phase 3) — `ApplySpatialStressSidecar(stress_spec, geom);`. After return, `geom.HasSAFSParams()` is true and `geom.sigma_n_per_dof()` is populated.
10. Resolve per-DOF friction params (Phase 1) — call `SpatialFrictionResolver::ResolveRateState(...)` using `geom.fault_dof_coords_3d()`, the bulk-element-owner indices (obtained from `ElasticityDomainOperator::GetFaultElemOwners()` — a small new accessor added in Phase 5, see below), `gmsk` attributes (constant 101 for the SAFS mesh, but kept general), the `MaterialField`, and `pmesh`. Get back `RateStatePerDOFParams` params. Combine with `geom.sigma_n_per_dof()` to populate `params.sigma_n_eff` (pore-pressure-adjusted).
11. Construct **`RateStateFaultOperator<ParMesh, 2>`** using the BP5 constructor that accepts a `FaultGeometry` and `BP5Params`. Then immediately call **`fault_op.SetSAFSMode(true, &tau_pre_per_dof, &sigma_n_per_dof)`** — where `tau_pre_per_dof` and `sigma_n_per_dof` are the vectors populated by `ComputeSAFSParams`. This is the wiring proved by `seas_test_safs_mode_wiring` (5/5 passing) and the new T_66_5 guard (R-001 in REVIEW.md fixes). The Phase-1 per-DOF `a`, `b`, `Dc`, `V_init`, `f_0`, `V_0`, `eta` are pushed into `RateStateFaultOperator` via new setters added in Phase 5.
12. Construct **`BP5SEASOp seas_op(&domain_op, &fault_op)`**; (alias for `SEASQuasiDynamicOperator<ParMesh, ElasticityDomainOperator<ParMesh>, BP5FaultOp>`).
13. Build initial state vector — `Vector state(seas_op.Width()); fault_op.PreInit(state);` (Phase 1: `V_init` from Phase 1 vector, ψ = steady-state). Then `seas_op.Solve(0.0, slip, u);` once to get domain-equilibrium traction. Then `fault_op.Init(state)` computes ψ from equilibrium per Phase 3 of the BP5 4-phase init.
14. Equilibrium verification — assert $\max_i |F(V_i, \psi_i)| < 1e-6 * V_init_max$ across all owned DOFs (`MPI_Allreduce`); abort otherwise with a per-rank list of offending DOFs.
15. If `--dry-run` — print the assembled summary (`L_nuc` per region, mesh stats, MPI layout, output paths) and exit 0.
16. Construct **`ParaViewOutput<ParMesh>`** — interior + shared fault face arrays from `domain_op.GetFaultInteriorFaces()` / `GetFaultSharedFaces()`. Call `pv.InitFaultOutputBP5(int_faces, shr_faces, nbf_per_face=1)`. Set fault output mode per `[output].paraview_fault`. Apply ZFP tolerances per TOML. Register displacement as the volume field.
17. Restart path — if `--restart PREFIX` set, call `PETScTSCheckpoint::Restore("<PREFIX>/checkpoint.h5", state, t0, dt0)` (existing). Otherwise `t0 = 0`, `dt0 = computed from L_nuc / V_nuc`.
18. Time loop — choose RK45 (`mfem::ODESolver` with `RKAdaptiveErrorEstimator`) or PETSc-TS based on `--petsc-ts`. Each step:
 - Adaptive Δt with `MPI_Allreduce(MPI_MAX)` on local error (CLAUDE.md BP5 v9 rule).
 - At each accepted step, `pv.UpdateFaultFieldsBP5(slip, slip_rate, traction, state_psi, sigma_n_eff)` and `pv.WriteIfDue(t)`.
 - Station probe writes per TOML cadence.
 - Every `checkpoint_every_steps` steps, write V2 PETSc-TS checkpoint via `PETScTSCheckpoint::Save(...)`.
19. Finalisation — final ParaView flush, final checkpoint write, MPI finalise.

Edge Cases to Handle

- Mesh is loaded but `ContainsBBox` fails on velocity sidecar — `LoadSpatialVelocityBundle` aborts with both bboxes printed. Driver does not catch.
- Friction TOML specifies **`slip_weakening`** law — driver aborts: “slip_weakening is only supported by `spatial_dyn_driver`; use `--config` with `[meta].law = rate_state` for `spatial_qd_driver`.” Cross-reference to sibling plan.

- **--restart PREFIX** and **<PREFIX>/checkpoint.h5** missing — abort with the exact path attempted.
- **--no-volume-pv** — sets [output].paraview_volume = "off" overriding TOML; documented as the default for BP5 production-style runs (saves disk).
- Sidecar present but **MaterialField::Mode::Coefficient** ctor not available (heterogeneous_material plan not yet merged) — abort at construction time with the missing-feature message: "ElasticityDomainOperator(MaterialField) constructor not found; check that heterogeneous_material_plan.md Phase 2 is merged."

Acceptance Criteria

- ☐ make seas_spatial_qd_driver builds; make test-spatial-qd-driver runs --dry-run successfully on the 1000m_lcfar3000 mesh + cvmh sidecar + stress sidecar + the EXAMPLE rate_state TOML, exits 0.
- ☐ mpirun -np 4 seas_spatial_qd_driver --config ... --tfinal 100s --dry-run builds the same configuration in parallel and exits 0.
- ☐ Equilibrium-verification step prints max |F| / V_init_max and passes the 1e-6 gate.
- ☐ V2 checkpoint round-trip: write at step 100, restart from that checkpoint, run 100 more steps; assert state at step 200 (continuous) == state at step 200 (restarted) to within ||state||_inf * 1e-12.
- ☐ Single-event smoke run (--tfinal 0.001yr) writes a non-empty volume.vtkhdf and fault.vtkhdf.

Dependencies

- Depends on: Phases 0-3; Phase 5 (operator accessors); heterogeneous_material_plan.md Phase 2; existing paraview + restart infrastructure.
- Required by: nothing (this is the leaf executable).

Phase 5: Operator accessor additions (minimal additive surface)

Goal

Add small accessor / setter methods to existing operators so the driver can wire per-DOF heterogeneous parameters without touching CLAUDE.md "do not modify" files.

Files to Modify

- **domain/elasticity_operator.hpp**: add

```
/// For each owned fault DOF, return the bulk element index (elem1 side)
/// that the SAFS friction resolver needs to query MaterialField::EvalAt.
/// Empty in BP2 / antiplane.
```

```
const Array<int>& GetFaultElemOwners() const;
```

populated alongside the existing per-DOF coords/basis computation.

- **fault/rate_state_fault.hpp**: add per-DOF parameter setters that populate the operator's internal vectors before Init is called. These are additive; if not called, the operator falls back to the BP5 scalar defaults exactly as today.

```
/// Override per-DOF friction parameters. Must be called BEFORE Init.
/// Each vector size must equal num_nodes_; partial overrides are not
/// supported (call with all five together). Aborts if SAFS mode is
/// not enabled (per-DOF parameters only make sense alongside per-DOF
/// tau_pre / sigma_n).
```

```
void SetPerDOFRateStateParams(const Vector& a,
                              const Vector& b,
                              const Vector& Dc,
                              const Vector& V_init_magnitude,
                              const Vector& f_0,
                              const Vector& V_0,
                              const Vector& eta);
```

Internally, `RateStateFaultOperator` switches its per-DOF read sites from `bp5_params_.a` (etc.) to the supplied vectors when present; same routing pattern as `safs_mode_` already uses for `tau_pre_per_dof_`.

Detailed Requirements

1. **GetFaultElemOwners** population: extend `ElasticityDomainOperator::InitFaultMaps` to record `fault_elem_owners[i] = local_elem_index_of_face_elem1(fault_face_index_of_dof_i)`. Already-existing internal `fault_face_to_elem` map suffices; just expose.
2. **SetPerDOFRateStateParams** semantics:
 - All seven vectors must have size `num_nodes_` (MFEM_VERIFY each).
 - Sets seven `per_dof_*` private vectors; sets `per_dof_params_enabled_ = true`.
 - All downstream reads of `bp5_params_.a` etc. become `(per_dof_params_enabled_ ? per_dof_a(i) : bp5_params_.a)`.
 - In `Mode::Constant` (`per_dof_params_enabled_ = false`, the default), every read site is bit-identical to today — pinned by an existing-test bit-exact regression.
3. No changes outside the two header additions above. No `.cpp` files for `rate_state_fault.hpp` (it's header-only). All changes live behind a new public setter; existing constructors and `PreInit/Init/ComputeRHS` signatures are unchanged.

Acceptance Criteria

- ☐ All existing BP5 tests (`test-bp5-fault-operator`, `test-bp5-integration`, `test-safs-mode-wiring`) pass bit-identically.
- ☐ New test `test_per_dof_rate_state_params` covers: (a) without calling setter → bit-identical to today; (b) calling setter with vectors that match `bp5_params_ scalars` → bit-identical; (c) calling setter with one DOF having a increased by 10% → `ComputeRHS` output at that DOF differs by the expected sensitivity.
- ☐ New test `test_get_fault_elem_owners` covers: BP5 mesh returns owners; antipplane returns empty.

Dependencies

- Depends on: nothing (purely additive accessors).
 - Required by: Phase 1's `ResolveRateState`, Phase 4's driver.
-

Phase 6: Sbatch + run script + minimal verification

Goal

Production-ready sbatch and a small Python verification script for a single-event smoke run.

Files to Create

- `jobs/safs/spatial_qd_smoke_8N_400r_dev_2hr_safs.sbatch` — Frontera dev queue, 8 nodes, 400 ranks, 2 hr wall. Builds `seas_spatial_qd_driver`, copies the 1000m_lcfar3000 mesh + cvmh + stress sidecars + EXAMPLE rate_state TOML into the sbatch's local work dir, runs `--tfinal 1yr --output-dir results_smoke_<jobid>`. Pattern mirrors `jobs/bp5/bp5_phase7_new_driver_dev_2hr.sbatch`.
- `jobs/safs/spatial_qd_production_normal_48hr_safs.sbatch` — Frontera normal queue, 32 nodes, full 48 hr, `--tfinal 100yr` with V2 restart every 5000 steps.
- `spatial/code/scripts/verify_spatial_qd_smoke_safs.py` — opens the resulting `fault.vtkhdf`, asserts max slip-rate is non-trivial ($\geq 1e-9$ m/s in any inter-seismic snapshot), peak coseismic slip rate $\geq 1e-3$ m/s if `--tfinal` is long enough to cover an event.

Detailed Requirements

1. sbatch must use the dedup macro path — `make seas_spatial_qd_driver` from sbatch will end with `$(SEAS_POST_LINK_DEDUP_RPATH)` per the Makefile R-005 convention, so the resulting Frontera binary will not duplicate `LC_RPATH`.
2. **`verify_spatial_qd_smoke_safs.py`**: `stdlib` + `h5py`; no `matplotlib` (per project preference for headless verification). Walks `<output_dir>/fault.vtkhdf` cycles, prints `max |V|` per snapshot, asserts the run wrote `>= 1` snapshot.
3. Document expected resource budget in a comment at the top of each sbatch (cores, peak RSS, wall, output volume in GB).

Acceptance Criteria

- ☐ Both sbatches lint-pass `sbatch --test-only`.
- ☐ `verify_spatial_qd_smoke_safs.py` runs cleanly on a previously-recorded reference `fault.vtkhdf` (committed under `tests/fixtures/safs_qd/`).

Dependencies

- Depends on: Phase 4.
- Required by: production runs (out of plan scope).

Testing Strategy

Layer	Test	When
TOML schema	<code>test_spatial_friction_config</code>	Phase 1 (10 tests)
Per-DOF resolver	<code>test_spatial_friction_resolve</code>	Phase 1 (8 tests)
Velocity bundle	<code>test_spatial_velocity_bundle</code>	Phase 2 (4 tests)
Stress bundle	<code>test_spatial_stress_bundle</code>	Phase 3 (3 tests)
Op accessors	<code>test_per_dof_rate_state_params</code> <code>test_get_fault_elem_owners</code>	Phase 5 (3 tests), (2 tests)

Layer	Test	When
Driver init	make test-spatial-qd-driver runs --dry-run (single configuration)	Phase 4
Restart	V2 checkpoint round-trip inside test-spatial-qd-driver umbrella	Phase 4
Smoke	verify_spatial_qd_smoke_safs.py on a 1-yr Frontera run	Phase 6

All new tests live in tests/unit/ (or tests/fixtures/ for golden references), use the existing TEST_ASSERT macro pattern, and end with \$(SEAS_POST_LINK_DEDUP_RPATH) in their Makefile link rule.

Risk Assessment

High-confidence (low risk)

- Phases 1-3 are pure additive Python-style C++: a TOML parser, a resolver, and two thin wrappers around already-tested infrastructure. No interaction with the critical numerical kernels.
- Phase 5 accessor additions are tiny: one getter on ElasticityDomainOperator, one setter group on RateStateFaultOperator. Both guarded by existing bit-exact regression tests.

Medium-risk

- **FaultGeometry::ComputePerDOFCoordsAndBasis_** zero-normal fallbacks on SAFS mesh. The REVIEW.md R-001 work showed that elasticity_operator-driven fixtures hit this. The SAFS multi-segment curvilinear fault may also hit it for DOFs at mesh corners or at the intersection of two SAFS strands. Mitigation: Phase 3's ApplySpatialStressSidecar aborts loudly via the R-001 guard; if it fires, the underlying ElasticityDomainOperator::GetFaultDOFBasis must be fixed before this driver can run. Investigation tag: // SPATIAL-QD R-001 TODO: investigate fault DOF basis on SAFS mesh.
- Per-DOF friction parameter coverage: the resolver applies rules in TOML order, last-match wins; a malformed TOML can leave large regions with default values (which validate but may be physically wrong). Mitigation: the --print-derived flag prints the histogram of a, b, Dc, etc. across DOFs so the user can sanity-check before launching a 48 hr run.

Low-confidence (need investigation before implementing)

- Whether the existing **SEASQuasiDynamicOperator<ParMesh, ElasticityDomainOperator<ParMesh>, BP5FaultOp>** template needs any change to support per-DOF η for the radiation-damping term. Currently BP5FaultOp uses scalar η . If the per-DOF wiring requires touching solver/seas_operator.hpp (CLAUDE.md "do not modify"), that's a plan amendment. Investigation: read solver/seas_operator.hpp:472 ComputeRHS call to confirm η is consumed via the fault operator's per-DOF accessor, not a scalar member.
- Pore-pressure spatial law. Right now the schema only supports constant + depth-gradient P_p. Real SAFS hydrology may want a sidecar field. Deferred to a follow-up plan unless the user provides a [pore_pressure.spatial] block in the TOML.
- Long-cycle restart determinism. V2 restart was proved on BP5 + TPV104 by the paraview-compaction merge. Whether it preserves bit-exactness across SAFS heterogeneous-material runs over 100+ years is unverified. Mitigation: Phase 4 acceptance includes a 100-step restart round-trip; production runs add the second --tfinal checkpoint test off-line.

Known tricky areas in existing code

- **FaultGeometry** BP5 constructor runs `ComputePerDOFCoordsAndBasis_` unconditionally. For SAFS this is the right path (per-DOF coords needed), but any silent fallback would corrupt downstream SAFS-mode reads — REVIEW.md R-001 guard already prevents this.
 - **RateStateFaultOperator::Init** 4-phase equilibrium init uses `bp5_params_.sigma_n` as the scalar baseline. With SAFS-mode on, the read switches to `sigma_n_per_dof(i)`. This is verified bit-exact for `SAFS_mode = false` by `seas_test_safs_mode_wiring T_66_2`; the on-side path is verified by `T_66_3 / T_66_4 / T_66_5`.
 - Paraview snapshot cap with adaptive cadence can throttle interseismic output in unexpected ways for SAFS (decade-scale runs). Re-tune `[output].max_snapshots` and `coseismic_dt` per-run by reading the post-run log line `[paraview] cap exhausted` (existing diagnostic).
-

Out of Scope for This Plan

1. Slip-weakening law on the quasi-dynamic path — the QD radiation-damping approximation pairs naturally with rate-and-state; LSW belongs to the dynamic-rupture sibling plan. This driver aborts if the friction TOML specifies `law = "slip_weakening"`.
2. Spatial pore-pressure sidecar. Constant + depth-gradient only.
3. Damage / time-dependent material. Material is static throughout the run.
4. Multi-fault interaction kinematics beyond what the `FaultGeometry` already supports — the SAFS multi-segment fault is treated as a single fault surface with attribute 101; per-segment friction rules are supplied via TOML spatial rules but no per-segment ψ -coupling beyond what the existing rate-state law produces naturally.
5. GPU offload — `MaterialField::EvalAt` in `Mode::Coefficient` is CPU-only (HDF5 trilinear), same constraint as the `heterogeneous_material` plan.
6. Wave-equation / dynamic-rupture path — see `spatial_dynamic_rupture_plan.md`.